The Problem of Overfitting

Overfitting is a common problem in machine learning where a model performs exceptionally well on the training data but fails to perform well on new, unseen data.

Key Concepts 🔑

- **Underfitting (High Bias):** The model is too simple and fails to capture the underlying patterns in the training data. It performs poorly on both the training set and new data.
    - **Bias** in this context refers to the model's strong, preconceived notion about the data (e.g., that the relationship must be linear), which prevents it from fitting the data well.
- **Overfitting (High Variance):** The model is excessively complex and learns the training data too well, including its noise and random fluctuations. It performs well on the training data but poorly on new data.
    - **Variance** refers to the model's sensitivity to small changes in the training data. A high-variance model can change drastically if the training data is slightly different.
- **Generalization:** This is the ultimate goal. A model is said to **generalize** well if it can make accurate predictions on new, unseen examples that were not part of its training set.
- **"Just Right":** The ideal model is one that is complex enough to capture the underlying trend of the data but not so complex that it overfits. It balances bias and variance and generalizes well.

Overfitting in Linear Regression (Housing Price Example)

Imagine fitting a model to predict house prices based on size:

1. **Underfitting (High Bias):** Fitting a simple straight line ($w_1 x + b$). The line fails to capture the trend that prices flatten out for larger houses.
2. **"Just Right":** Fitting a quadratic function ($w_1 x + w_2 x^2 + b$). This curve captures the general trend of the data well and is likely to generalize to new houses.
3. **Overfitting (High Variance):** Fitting a high-order (e.g., 4th-order) polynomial. The curve becomes excessively "wiggly" to pass through every single data point perfectly. The cost function might be zero, but it makes nonsensical predictions for new data points.

Overfitting in Classification (Tumor Example)

The same concepts apply to classification tasks like predicting if a tumor is malignant:

1. **Underfitting (High Bias):** Using a simple logistic regression model with a linear decision boundary. The straight line is unable to effectively separate the two classes.
2. **"Just Right":** Adding quadratic features to create a more flexible, curved decision boundary (like an ellipse). This model separates the classes well without being overly complex.
3. **Overfitting (High Variance):** Using very high-order polynomial features. The model creates an overly complex, contorted decision boundary that perfectly separates all training examples but is unlikely to generalize well to new patients.

Addressing Overfitting 🛠️

When a model is identified as being **overfit** (having high variance), there are several strategies you can employ to improve its ability to generalize to new data.

---

1. Collect More Training Data

- **How it works:** Providing the algorithm with more examples helps it learn the true underlying patterns in the data, rather than fitting to the noise in a small dataset. A larger dataset can naturally smooth out a complex, "wiggly" curve.
- **Effectiveness:** This is often the most reliable way to combat overfitting.
- **Limitation:** It's not always possible or practical to acquire more data.

---

2. Use Fewer Features (Feature Selection)

- **How it works:** Overfitting can be caused by having too many input features relative to the amount of data. By selecting a subset of the most relevant features, you can reduce the model's complexity.
- **Methods:**
  - **Manual Selection:** Using your domain knowledge and intuition to choose the most important features.
  - **Automated Selection:** Using algorithms (discussed in later courses) to automatically identify the most useful features.
- **Disadvantage:** You risk **throwing away useful information** by discarding features that might have some predictive value.

---

3. Regularization

- **How it works:** This technique keeps all the features but reduces the magnitude (size) of the parameters $\vec{w}$.
- **Intuition:** Overfitting often corresponds to very large parameter values, which create highly complex and sensitive models. Regularization adds a penalty to the cost function for large parameters, encouraging the learning algorithm to find smaller values for them.
- **Effect:** This results in a simpler, smoother model that is less likely to overfit. It's a "gentler" approach than eliminating features entirely.
- **Convention:** Regularization is typically applied to the weight parameters ($w_1, w_2, \ldots, w_n$) but not to the bias parameter (b).

Cost Function with Regularization

Regularization works by modifying the cost function to penalize large parameter values, which encourages the model to be simpler and less prone to overfitting.

The Core Idea: Penalizing Large Parameters

- **Problem:** In an overfit model (e.g., a high-order polynomial), the parameter values (w_j) are often very large, leading to a complex, "wiggly" function.
- **Solution:** We can modify the cost function to not only minimize the error on the training data but also to keep the parameter values small.
- **Example:** To simplify a 4th-order polynomial, we could add penalty terms like $+1000w_3^2 + 1000w_4^2$ to the cost function. To minimize this new cost, the algorithm is forced to choose very small values for w_3 and w_4, effectively making the model behave more like a simpler quadratic function.

The Regularized Cost Function

Instead of penalizing specific parameters, regularization is typically applied to *all* weight parameters, w_j.

- The original cost function is modified by adding a **regularization term**:
  $$J(\vec{w}, b) = \underbrace{\frac{1}{2m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})^2}_{\text{Original Cost}} + \underbrace{\frac{\lambda}{2m} \sum_{j=1}^{n} w_j^2}_{\text{Regularization Term}}$$
- **Components:**
  - **Original Cost:** This part encourages the model to fit the training data well.
  - **Regularization Term:** This part encourages the parameters (w_j) to be small, which helps prevent overfitting.
  - **λ (Lambda):** The **regularization parameter**. This is a value you choose that controls the trade-off between the two goals.
- **Convention:**
  - The bias parameter, b, is typically **not** regularized.
  - The term is scaled by $\frac{1}{2m}$ to make the math cleaner and help keep λ consistent even if the training set size changes.

The Role of the Regularization Parameter (λ) ⚖️

The value of λ determines the strength of the regularization penalty and critically affects the model's final behavior.

- **If $\lambda = 0$:** The regularization term becomes zero. The model is not regularized at all and will likely **overfit**.
- **If λ is very large (e.g., 10^{10}):** The penalty for large parameters is huge. To minimize the cost, the algorithm will force all w_j parameters to be extremely close to zero. The model becomes too simple (e.g., $f(\vec{x}) \approx b$), resulting in a flat line that **underfits** the data.
- **If λ is "just right":** The model finds a good balance between fitting the data and keeping the parameters small, resulting in a model that **generalizes well**.

Gradient Descent for Regularized Linear Regression

This section explains how to adapt the gradient descent algorithm to work with the new, regularized cost function for linear regression.

The Regularized Cost Function (Recap)

The cost function for regularized linear regression includes an additional term to penalize large parameter values:

$$J(\vec{w}, b) = \frac{1}{2m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m} \sum_{j=1}^{n} w_j^2$$

Updated Gradient Descent Algorithm

To minimize this new cost function, we need to update the partial derivative terms used in the gradient descent update rules.

- **Derivative with respect to w_j:**
  - The original derivative is modified by an additional regularization term.
  - $\frac{\partial J}{\partial w_j} = \left(\frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)} \right) + \frac{\lambda}{m} w_j$
- **Derivative with respect to b:**
  - Since the bias term b is not regularized, its derivative remains **unchanged**.
  - $\frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})$

Implementation of the Update Rules 👨‍💻

The complete gradient descent algorithm for regularized linear regression involves repeatedly performing the following **simultaneous updates**:

- **Update for w_j:**
  $w_j := w_j - \alpha \left[\left(\frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)} \right) + \frac{\lambda}{m} w_j \right]$
- **Update for b:**
  $b := b - \alpha \left[\frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) \right]$

Intuition Behind the Update (Optional)

We can rewrite the update rule for w_j to better understand what regularization is doing on each step:

$$w_j := w_j \left(1 - \alpha \frac{\lambda}{m}\right) - \alpha \frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)}$$

- The term $\left(1 - \alpha \frac{\lambda}{m}\right)$ is a number that is typically just **slightly less than 1** (e.g., 0.999).
- This means that on every iteration, before subtracting the usual gradient term, the algorithm first **multiplies w_j by a number less than one**, which has the effect of **shrinking the weight** slightly.
- This constant shrinking effect is how regularization prevents the parameters from growing too large.

Regularized Logistic Regression

Just like linear regression, logistic regression can be regularized to prevent overfitting, especially when using a large number of features or high-order polynomial terms.

The Problem: Overfitting in Logistic Regression

When logistic regression is trained with many features, it can create an overly complex decision boundary that fits the training data perfectly but fails to generalize to new data.

The Regularized Cost Function

To address overfitting, we add a regularization term to the logistic regression cost function. This penalizes large values for the weight parameters (w_j).

- **The final cost function for regularized logistic regression is:**
  $J(\vec{w}, b) = \left[-\frac{1}{m}\sum_{i=1}^{m} y^{(i)} \log\left(f_{\vec{w},b}(\vec{x}^{(i)})\right) + (1 - y^{(i)}) \log\left(1 - f_{\vec{w},b}(\vec{x}^{(i)})\right)\right] + \frac{\lambda}{2m}\sum_{j=1}^{n} w_j^2$
- Adding this term encourages the algorithm to find smaller parameter values, resulting in a simpler, smoother decision boundary that is less likely to overfit.

Gradient Descent Implementation 👨‍💻

The gradient descent algorithm is updated to account for the new regularization term in the cost function.

- **Partial Derivatives:**
  - The derivative with respect to w_j gets an additional term:
    $\frac{\partial J}{\partial w_j} = \left(\frac{1}{m}\sum_{i=1}^{m}(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})x_j^{(i)}\right) + \frac{\lambda}{m}w_j$
  - The derivative with respect to b remains **unchanged**, as the bias term is not regularized.
- **Simultaneous Update Rules:**
  - **Update for w_j:**
    $w_j := w_j - \alpha\left[\left(\frac{1}{m}\sum_{i=1}^{m}(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})x_j^{(i)}\right) + \frac{\lambda}{m}w_j\right]$
  - **Update for b:**
    $b := b - \alpha\left[\frac{1}{m}\sum_{i=1}^{m}(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})\right]$
- **Key Similarity:** The update rule equations are **identical** to those for regularized linear regression. The only difference is the definition of $f_{\vec{w},b}(\vec{x})$, which is the sigmoid function for logistic regression.